

Lightweight Privacy-Preserving IDS for ITS: Integrated Federated Learning and Blockchain

Jiawei Zha¹, Guoan Zhang², *Senior Member, IEEE*, Li Jin³, Shuping Dang⁴, *Senior Member, IEEE*,
and Wei Duan⁵

Abstract—As intelligent transportation systems (ITS) become more integrated into modern infrastructure, vehicular communication systems face increasing cybersecurity threats. Traditional centralized intrusion detection system (IDS) has significant limitations in terms of the scalability, privacy preservation, and trust establishment, which are critical challenges in ITS environments. To address these issues, this paper proposes an intrusion detection method tailored specifically for ITS, integrating federated learning (FL) and blockchain technology to create a secure, scalable, and privacy-preserving threat detection system for the Internet of Vehicle (IoV). The proposed method utilizes FL for distributed model training, avoiding the sharing of raw data and employing encryption techniques to protect user privacy at the edge devices. Blockchain technology ensures the integrity and tamper-proof nature of model updates and fosters trust between entities. Moreover, to accommodate the heterogeneous and dynamic data environment in IoV, the method supports both independent and identically distributed (IID) and non-IID data scenarios, enhancing the system's adaptability and robustness. Given the computational limitations of vehicular devices, this work incorporates knowledge distillation and lightweight model designs, effectively reducing the local computational burden. Experimental results demonstrate the significant effectiveness of the proposed approach: on the CICIDS2017 dataset, the model achieves an accuracy of 97.10% with 11,904 parameters and a memory consumption of 0.045MB; on the Car-Hacking dataset, it achieves an accuracy of 99.10% with 9,989 parameters and a memory consumption of 0.038MB; on the CICIoV2024 dataset, it achieves an accuracy of 99.65% with 9,861 parameters and a memory consumption of 0.038MB. Compared to traditional baseline models, the proposed approach significantly reduces both the number of training parameters and memory consumption, while maintaining high accuracy, making it highly suitable for deployment in resource-constrained vehicular environments within ITS.

Index Terms—Federated learning, blockchain, knowledge distillation, intrusion detection, IoV.

I. INTRODUCTION

WITH the development and increasing sophistication of the Internet of Vehicle (IoV) [1], [2], [3], vehicles are facing an escalating array of cybersecurity threats, including hacker attacks, malware, and unauthorized access. These threats will jeopardize the functionality of vehicles and the safety of passengers. As a result, the robust intrusion detection system (IDS) has become an urgent necessity in IoV [4], [5], [6], [7]. The primary objective of IDS is to safeguard the integrity and security of both onboard and external networks by addressing increasingly complex threats. In IoV, IDS implements multi-layered defenses, involving access control mechanisms to prevent unauthorized access, and comprehensive traffic analysis to identify potential attack behaviors. For example, IDS deployed at network gateways can intercept malicious packets before they propagate, while in-vehicle IDS is normally integrated within the controller area network (CAN) [8], [9], [10], electronic control units (ECUs), and various sensors to provide comprehensive protection.

However, as vehicle interconnectivity continues to increase, IDS also evolves, offering adaptive solutions to emerging threats and ensuring robust security for modern automotive systems. By leveraging artificial intelligence technologies (AI), IDS can adaptively learn and update attack signatures, thereby improving detection accuracy and efficiency, meanwhile enhancing its adaptive capabilities [11], [12], [13]. In recent years, machine learning-based intrusion detection methods have been widely researched and applied in intelligent transportation systems. Yang et al. [14] proposed a multitiered hybrid IDS that integrates both signature-based and anomaly-based detection mechanisms to address known and unknown attacks in vehicular networks. Lu et al. [15] developed a deep convolutional factorization machine-based IDS for CAN bus data. It captured both local features and interaction information between high-order and raw binary features, achieving the precision, accuracy, and F1 scores of 0.868, 0.918, and 0.923, respectively. Sayem et al. [16] presented a deep learning-based ensemble framework, including dynamic preprocessing, optimal feature selection, and handling of imbalanced data. It achieved 90.6% and 99.6% accuracy on two publicly available

Received 16 August 2025; revised 12 December 2025; accepted 29 January 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62471258 and Grant 62571278, in part by the Key Research and Development Program of Nantong under Grant GZ2024002, in part by Nantong Municipal Science and Technology Program for Social Livelihood under Grant MS2024068, and in part by the Natural Science Foundation of Nantong under Grant JC2024103. The Associate Editor for this article was X. Li. (*Corresponding authors: Guoan Zhang; Li Jin.*)

Jiawei Zha, Guoan Zhang, Li Jin, and Wei Duan are with the School of Information Science and Technology, Nantong University, Nantong 226019, China (e-mail: 2330320030@stmail.ntu.edu.cn; gzhang@ntu.edu.cn; jin.li@ntu.edu.cn; sinder@ntu.edu.cn).

Shuping Dang is with the School of Electrical, Electronic and Mechanical Engineering, University of Bristol, BS8 1UB Bristol, U.K. (e-mail: shuping.dang@bristol.ac.uk).

Digital Object Identifier 10.1109/TITS.2026.3661633

and popular network traffic datasets, namely UNSW-15 and CICIDS2017. Zhang et al. [17] introduced a transformer-based transfer learning system, which analyzed spatiotemporal sequences, along with a self-supervised box classification mechanism to enhance detection of unknown threats. Wu et al. [18] proposed an intelligent IDS combining fuzzy rough sets, generative adversarial network (GAN), and convolutional neural network (CNN), leveraging big data mining and efficient feature extraction for intrusion detection.

Despite such advances, IDS still faces significant privacy and trust challenges in the IoV. Vehicle-generated data inherently contains sensitive user information, and the lack of effective privacy protection not only undermines trust but also increases the risk of non-compliance with data regulations. This absence of privacy protection further complicates the establishment of trust between onboard units, cloud servers, and security centers, creating barriers to effective collaboration. To address such issues, several studies have applied federated learning (FL) for intrusion detection in certain fields. Cui et al. [19] designed a collaborative intrusion detection system (CIDS), using FL for collaborative model training among software-defined networking (SDN) nodes. Liu et al. [20] developed a delay and energy-efficient asynchronous FL framework for intrusion detection for heterogeneous industrial internet of things (IIoT) environments. Zhang et al. [21] proposed an anomaly-based federated IDS with a hierarchical aggregation scheme using weighted voting. Aouedi et al. [22] introduced a federated semi-supervised approach leveraging labeled and unlabeled data collaboratively. However, without encryption, FL is still vulnerable to gradient leakage.

Meanwhile, to improve IDS accuracy, increasingly complex deep learning models have been developed. However, their deployment in vehicular environments is constrained by severe resource limitations. In the IoV, ECUs are with limited computing power, memory, and energy, making it difficult to run large models in real-time. Lightweight deep learning models aim to reduce complexity and speed up inference through simpler structures and efficient computations, while maintaining acceptable accuracy.

Motivated by these challenges, we propose a lightweight privacy-preserving IDS based on blockchain and FL. Unlike our previous work [23], which enhanced the performance of IDS in IoV by leveraging knowledge graphs and machine learning techniques to improve detection capabilities against various types of attacks, our previous work conducted intrusion detection on a centralized server, and utilized shapley additive explanations (SHAP) for interpretability analysis to enhance the transparency of the model's decision-making process. Our main contributions of this work are summarized as follows.

- This paper introduces an intrusion detection approach specifically designed for IoV, integrating blockchain, FL, and advanced AI models to ensure the security of vehicular data. By leveraging the distributed ledger characteristic of blockchain, data integrity and transparency are guaranteed. The application of FL addresses issues related to data privacy and decentralized storage, enabling collaborative training among vehicles without sharing raw data or compromising security.

- Constructing a domain-specific knowledge graph from vehicular IDS datasets by transforming raw traffic records into semantically rich entities and relationships. This graph-based structured representation not only facilitates advanced feature extraction, enabling the capture of high-order dependencies and contextual correlations, but also significantly reduces redundancy and noise in the input data, leading to substantially improved intrusion-detection accuracy and robustness.
- Our work employs a model compression technique, specifically knowledge distillation, to transfer knowledge from a complex teacher model to a more compact and efficient student model. The process of transferring knowledge enables the student model to leverage the representations and decision-making capabilities learned by the teacher model, while maintaining a reduced computational overhead.
- We propose a federated model parameter encryption and decryption mechanism based on CRT, and employ parallel batch encryption, which enables lightweight and parallelizable protection of model updates without relying on heavyweight cryptographic primitives such as rivest-shamir-adleman (RSA) or homomorphic encryption (HE). Furthermore, it incurs much lower overhead in terms of encryption and decryption time.

The remainder of this paper is organized as follows. Section II presents the preliminary background. Section III details our proposed methodology. Section IV provides the experimental evaluation. Finally, section V concludes this paper.

II. PRELIMINARIES

In this section, we present the foundational concepts underlying the techniques employed in this study, including CRT, blockchain, and FL.

A. Chinese Remainder Theorem

CRT, which was first appeared in the 3rd century in the work *Sunzi Suanjing* by the Chinese mathematician Sunzi, in its modern form, CRT has been widely used in areas such as computer science, cryptography, parallel computing, and error-correcting codes [24].

The theorem primarily addresses the solution of systems of simultaneous linear congruences of the following type:

$$\begin{cases} x \equiv a_1 \pmod{m_1}, \\ x \equiv a_2 \pmod{m_2}, \\ \vdots \\ x \equiv a_k \pmod{m_k}, \end{cases} \quad (1)$$

when the moduli $m_1, m_2, \dots, m_k$ are pairwise coprime, the above system has a unique solution modulo $M = m_1 m_2 \cdots m_k$, and its solution can be explicitly constructed as:

$$x \equiv \sum_{i=1}^k a_i M_i y_i \pmod{M}, \quad (2)$$

where $M_i = \frac{M}{m_i}$, and y_i is the modular inverse of M_i modulo m_i , satisfying

$$M_i y_i \equiv 1 \pmod{m_i}. \quad (3)$$

Clearly, this formulation guarantees both the existence and uniqueness of the solution, providing an efficient computational method. By allowing computations to be carried out in parallel over smaller moduli and then recombined, CRT significantly enhances computational efficiency.

B. Blockchain

Blockchain is a decentralized, transparent, tamper-resistant distributed ledger technology. Initially proposed by Satoshi Nakamoto in 2008 to support the bitcoin system, it leverages cryptographic techniques to eliminate the need for trusted centralized institutions, enabling the immutable recording and updating of information across untrusted participants.

A blockchain is composed of a series of blocks connected in chronological order. Each block typically consists of the following components:

- Hash value of the previous block H_{i-1} ,
- Timestamp T_i ,
- Random number N_i ,
- Merkle root MT_i ,
- Block body $\{TX_1, TX_2, \dots, TX_n\}$, containing a list of transactions.

The complete data content of block B_k , denoted as D_k , includes all transactions and metadata (e.g., timestamp, merkle root),

$$D_k = \{TX_1, TX_2, \dots, TX_n, T_k, N_k, MT_k\}, \quad (4)$$

where each block's hash value H_i is computed from its contents as:

$$H_i = \Gamma(H_{i-1} \parallel D_k), \quad (5)$$

with Γ as a cryptographic hash function i.e., SHA-256, and $\parallel$ as the concatenation.

Because each block includes the hash of the previous block, the blocks are chained together. Thus, any modification to one block will affect all subsequent blocks' hash values, ensuring the immutability of overall system.

If an attacker attempts to tamper with the data D_k in block B_k , then all hash values from H_k onward should be recomputed to preserve the chain's integrity, otherwise, the chain becomes invalid. For instance, if the data is modified as D'_k , then the new hash value becomes:

$$H'_k = \Gamma(H_{k-1} \parallel D'_k) \neq H_k. \quad (6)$$

Before being tampered, the hash value of the next block is determined by the current block, obtained from:

$$H_{k+1} = \Gamma(H_k \parallel D_{k+1}). \quad (7)$$

Then the modified H_{k+1} can be defined as:

$$H'_{k+1} = \Gamma(H'_k \parallel D_{k+1}) \neq H_{k+1}. \quad (8)$$

Because $H'_k \neq H_k$, $H'_{k+1} \neq H_{k+1}$ also holds. By induction, for all $i \geq k$, we have $H'_i \neq H_i$. That is, all subsequent hash values are altered. The above process demonstrates the robust security of blockchain in safeguarding data integrity and privacy.

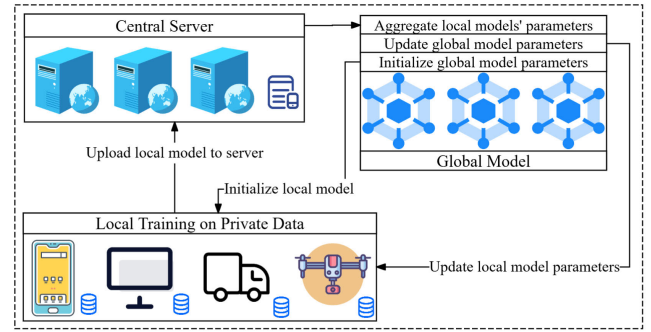


Fig. 1. Fundamental workflow of FL.

C. Federated Learning

FL is a distributed collaborative training framework in which each participant (client) retains its local data and collaboratively trains a global model through periodic exchange of model parameters, rather than sharing raw data directly.

The main process is illustrated in Fig. 1, which can be summarized as follows:

- 1) The central server initializes the global model parameters upon startup.
- 2) Each client downloads the global model and performs local training using its private data to obtain local model updates each round.
- 3) All local model parameters are uploaded to the central server for aggregation.
- 4) The server aggregates the local parameters (e.g., via weighted averaging) to obtain an updated set of global model parameters for the next training round.

III. PROPOSED METHOD

The proposed scheme consists of eight stages: real-world attack scenario, system model, data preprocessing, model distillation, local model training, parameter encryption, global model aggregation, and local model updating.

A. Real-World Attack Scenario

As shown in Fig. 2, this section presents specific attack scenarios that vehicles may encounter in intelligent transportation systems (ITS). Hackers can directly infiltrate vehicles and control their in-vehicle networks (e.g., controller area network, CAN), or they can penetrate further by attacking various communication links associated with the vehicle (i.e., the vehicle's external networks), ultimately gaining control of the vehicle. Once a vehicle is compromised, it may become a malicious node and launch attacks on other vehicles, posing a significant threat to public safety. In the following, we will analyze the various attacks depicted in the figure in details.

V2V link attack: Hackers can disrupt or hijack the communication link between vehicles (V2V), allowing them to forge or alter information transmitted between vehicles. Such attacks could lead to the failure of collaboration between vehicles, causing traffic accidents or operational errors, which

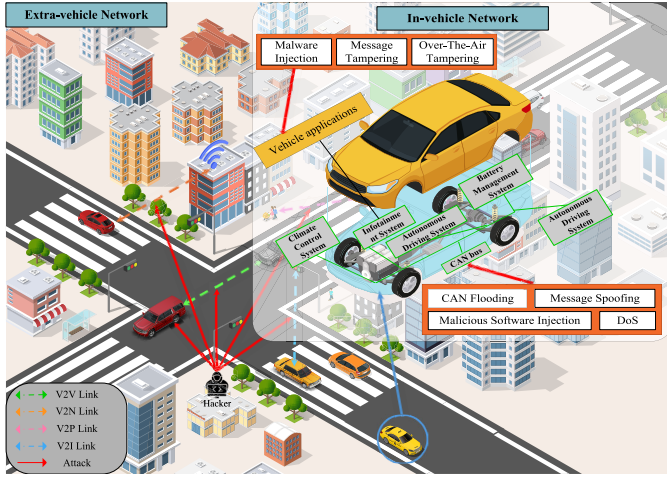


Fig. 2. The real-world attack scenario for IoV in ITS.

in turn affects vehicle coordination and cooperation, severely compromising traffic safety.

V2I link attack: Hackers can attack the communication link between vehicles and infrastructure (V2I), such as by tampering with traffic signals or accessing traffic control information. This type of attack can disrupt traffic control systems, potentially causing system crashes or operational errors, which can affect the normal operation of the entire traffic network and pose significant public safety risks.

V2N link attack: Hackers can communicate with vehicles through the network link (V2N), forging or altering commands from external networks, leading to faults in the vehicle control systems or the leakage of sensitive data. This type of attack could result in the loss of vehicle functionality or data breaches, threatening vehicle safety and user privacy.

V2P link attack: Hackers can compromise the communication link between vehicles and pedestrians (V2P), potentially endangering pedestrian safety or causing misunderstandings or incorrect responses during interactions between vehicles and pedestrians. This attack not only jeopardizes pedestrian lives but could also negatively impact the normal functioning of the traffic system.

In-vehicle attack: Attackers can target the vehicle's CAN by sending large amounts of invalid or malicious data, causing the vehicle control system to overload. As a result, the vehicle may experience system failures or loss of control, leading to more severe traffic accidents.

Consequently, in the context of in-vehicle networks, IDS can be deployed on the ECU or the in-vehicle gateway to monitor communication data flows within the vehicle's internal network, identifying potential malicious attacks. For IDS deployment in external vehicular networks, it can be positioned at locations such as road side units (RSUs), V2V communication nodes, and cloud-based security centers, where it can continuously monitor and analyze network traffic in real-time to detect anomalous behaviors or attacks.

B. System Model

In light of the requirements of the above intelligent transportation system, we propose an intrusion detection method,

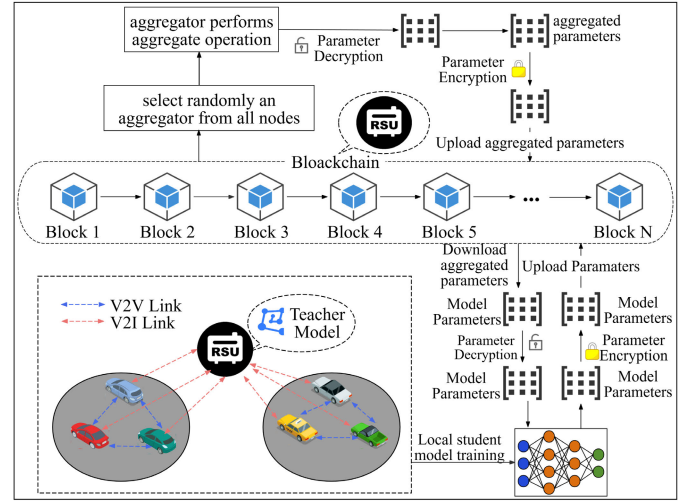


Fig. 3. System architecture of the proposed scheme.

as illustrated in Fig. 3. To protect vehicular privacy data, we employ a FL distributed framework, where local training is performed on individual vehicles. Once the teacher model training is completed, the vehicles perform the training of the student model locally. After training, the model parameters are uploaded and stored on blockchain to prevent tampering. Each block generates a unique hash value: if a malicious user attempts to alter the model parameters, the hash value will change, thereby ensuring integrity. Then, a randomly selected aggregator performs parameter aggregation for the current training round, producing a unified set of model parameters. These aggregated parameters are subsequently uploaded back to the blockchain, after which local models retrieve the updated parameters and retrain until optimal accuracy is achieved. The following descriptions provide detailed explanation of the content in the figure:

- **RSU:** As a central node in the network, the RSU is responsible for aggregating and coordinating data transmission between different vehicles and may also act as an aggregator. The RSU can also maintain the blockchain to verify and record all model updates.
- **Teacher Model:** The teacher model is used in the RSU for model management, acting as a mechanism to transfer knowledge. It serves as a guide for other vehicles (student models), allowing them to obtain knowledge from the teacher model for better training outcomes.
- **Blockchain:** In the context of ITS, blockchain technology is introduced primarily to address the trust establishment and secure collaboration challenges in the distributed vehicular networks. In an open environment consisting of numerous vehicles and roadside units, ensuring the immutability and traceability of key information such as model updates is crucial. Blockchain, with its decentralized, transparent, and tamper-resistant distributed ledger properties, provides a trustworthy collaborative platform for all participants. Any updates to model parameters are encrypted and chained in the blockchain. If any malicious node attempts to alter the data, its behavior

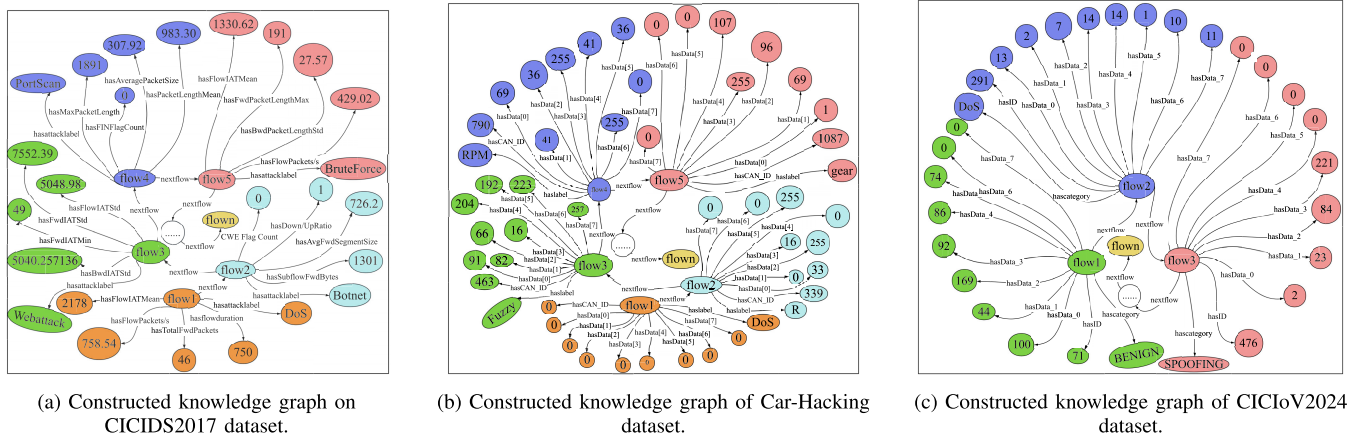


Fig. 4. Constructed knowledge graph on CICIDS2017, Car-Hacking and CICIoV2024 dataset.

will be immediately detected due to the breakage of the hash chain, effectively preventing model poisoning attacks. This creates a trust mechanism that does not rely on a central authority, which is crucial for the safety-critical systems of intelligent transportation. In section II, we provide a specific analysis and proof of blockchain's immutability feature, which ensures data integrity and trust within the system.

- **Lock:** The yellow lock icon in the figure represents the encryption operation, ensuring that data remains secure and is not tampered with or accessed by unauthorized entities during the process of uploading and downloading model parameters to and from the blockchain. The gray lock icon represents the decryption operation, used for decrypting during global parameter aggregation and vehicle parameter updates.

C. Data Preprocessing

First, we preprocess the datasets related to IoV, including Car-Hacking, CICIoV2024, and CICIDS-2017, which contain normal network traffic data and various types of attack data. These datasets include some missing values and outliers. The preprocessing procedure involves data cleaning, feature selection, followed by knowledge graph construction, and finally, standardization.

1) **Data Cleaning:** Raw data containing missing values or outliers is removed to ensure that the processed dataset is free of such irregularities.

2) **Feature Selection:** We employ a machine learning model to assess the importance of features and select the top 23 features based on their importance ranking, along with their corresponding label columns.

3) **Knowledge Graph Construction:** Vehicular network attacks often exhibit contextual associations and higher-order dependencies, which traditional feature vectors struggle to effectively capture. The construction of the knowledge graph is primarily aimed at significantly enhancing the deep semantic understanding of complex and intricate attack patterns in IDS. Prior to constructing the knowledge graph, data cleaning and feature selection are performed to remove redundant and

irrelevant features, thereby reducing the dimensionality of the dataset. Subsequently, we construct the datasets into a knowledge graph using the resource description framework (RDF). Knowledge graphs possess strong expressive capabilities for modeling attack categories and network behavior characteristics, effectively capturing high-order correlations in attack patterns. By leveraging rich semantic representations, knowledge graphs can automatically uncover latent relationships between features.

Specifically, knowledge information is stored in the form of RDF triples (s, p, o) , where s represents the subject, used to identify the entity or concept being asserted, p symbolizes the predicate, used to describe the relationship or attribute between the subject and object, and o means the object, indicating the value or entity connected to the subject through the predicate. In the process of constructing RDF triples, we first use `rdflib.Graph()` to create an RDF graph, and then define a namespace to uniformly identify entities and properties. Subsequently, we iterate through each row of the dataset, defining each piece of network traffic data as an RDF entity and adding traffic features as the corresponding properties. Additionally, an attack type (label) is assigned as a property of each record in the dataset. For example, each row of the preprocessed dataset is treated as a network flow and assigned a unique identifier, such as the entity *flow1*. If it is identified as a DoS attack, it can be represented as $(flow1, is, DoS)$, or if its *Flow IAT Mean* is X , it is represented as $(flow1, hasFlowIATMean, X)$. The triple representation forms the basic structure of the knowledge graph, facilitating the structured organization and retrieval of interconnected information. Finally, all generated triples are added to the graph for the construction of the knowledge graph. We visualize the constructed knowledge graph, as shown in Fig. 4(a). Furthermore, the knowledge graph shown in Fig. 4(a) does not encompass all three datasets used in this study. It is specifically the knowledge graph constructed from the CICIDS2017 dataset. In order to ensure the clarity and readability of the content in the figure, we only present a portion of it. The knowledge graphs constructed from the other two datasets, namely the Car-Hacking and CICIoV2024 datasets, are shown in Fig. 4(b-c). Similarly, data

cleaning process is initially performed, with the distinction that no feature selection was conducted for the Car-Hacking and CICIoV2024 datasets. We extracted features such as CAN ID, packet size, and vehicle sensor data, and subsequently transformed them into RDF triples. Following this, knowledge graphs are constructed.

4) *Standardization*: Different features may have varying scales and magnitudes (e.g., port numbers may range in the thousands, while packet sizes may reach the millions). Without standardization, certain features may disproportionately influence gradient updates during model training, leading to oscillations or even failure to converge. For instance, in neural networks, unnormalized data can easily result in vanishing or exploding gradients. Therefore, we apply min-max normalization, calculated as follows:

$$X = \frac{x - x_{\min}}{x_{\max} - x_{\min}}. \quad (9)$$

D. Model Distillation

With the continuous limitations in computational resources, memory, and power consumption of IoV devices, especially in real-time intrusion detection tasks that require processing large amounts of network traffic data, running traditional large deep learning models becomes extremely challenging. To effectively address this issue, this paper proposes a lightweight model design method based on knowledge distillation to alleviate the computational burden on vehicular devices and improve model efficiency. Therefore, for the local models, we adopt a model compression technique known as knowledge distillation, which transfers knowledge from a complex teacher model to a smaller, more efficient student model. The teacher model is usually trained on computationally abundant cloud servers or RSUs, while the student model is deployed on vehicular devices with limited computational resources. In this way, the student model not only learns the high-level feature representations and decision-making abilities embedded in the teacher model but also maintains a low computational and storage demand, ensuring efficient operation on vehicular devices.

The knowledge distillation process is outlined in Algorithm 1. During the knowledge distillation process, the temperature parameter is set to 5.0. This parameter controls the smoothness of the student model's output, enhancing the model's learning capability. And the weight ratio between the distillation loss and the true label loss (i.e., alpha) is initially set to 0.8 to enhance the model's learning ability while minimizing the computational burden on resource-constrained vehicles. As training progressed, the alpha value gradually decreases to 0.3 to balance the distillation loss and the student model's training on the true labels. Moreover, we implement two callback functions:

- *ReduceLROnPlateau*: This strategy dynamically adjusts the learning rate by monitoring the training loss. If the loss fails to decrease for two consecutive epochs, the learning rate is halved, with a lower bound set at 1×10^{-6} .
- *EarlyStopping*: An early stopping mechanism that halts training if the loss shows no improvement for three

Algorithm 1 Knowledge Distillation

Input: Teacher model f_T , training data X , training labels y , epochs E , batch size B , temperature T , loss weight α

Output: Trained student model f_S

- 1: Get teacher logits $t \leftarrow f_T(X)$
- 2: Define callbacks:

callbacks = [
 ReduceLROnPlateau(monitor='loss', factor=0.5,
 patience=2, min_lr=1e-6, verbose=True),
 EarlyStopping(monitor='loss', patience=3, restore_best_weights=True, verbose=True),
]
- 3: **for** epoch $\epsilon \in (1, E + 1)$ **do**
- 4: **for** batch $\epsilon \in (0, \text{len}(X), B)$ **do**
- 5: Get current batch:
- 6: $X_batch = X[\text{batch: batch} + B]$
- 7: $y_batch = y[\text{batch: batch} + B]$
- 8: $t_batch = t[\text{batch: batch} + B]$
- 9: Get Student predictions: $s \leftarrow f_S(X_batch)$
- 10: Calculate distillation loss:
- 11: $distill_loss \leftarrow \text{KL}(\text{softmax}(t_batch/T), \text{softmax}(s/T)) \cdot (T^2)$
- 12: Calculate student loss: $student_loss \leftarrow \text{CCE}(y_batch, s)$
- 13: Combined loss: $total_loss \leftarrow \alpha \cdot student_loss + (1 - \alpha) \cdot distill_loss$
- 14: Update the student model f_S through $total_loss$ and gradients
- 15: Weight pruning for the student model f_S
- 16: **end for**
- 17: **end for**
- 18: **return** f_S

consecutive epochs, automatically restoring the model weights from the epoch with the lowest loss.

Additionally, we employ the neural network weight pruning method (*apply_pruning*), whose core functionality is to reduce the number of effective parameters in the model by setting weights with absolute values smaller than a specified threshold to zero, thereby achieving model compression and acceleration. This method accepts an optional parameter, *pruning_threshold* (default value: 0.01), to control the intensity of pruning: a higher threshold leads to more aggressive pruning (more weights are zeroed out), while a lower threshold results in more conservative pruning (retaining more weights).

E. Local Model Training

Within the framework of IoV, the data generated by vehicles contains a substantial amount of sensitive information, which is subject to stringent data protection regulations. Traditional centralized processing methods present a significant risk of privacy leakage. Therefore, to address the privacy and distributed characteristics of data in ITS, we employ the FL framework for local models. In this approach, local vehicles only need to upload model parameters rather than raw data, fundamentally preventing data from leaving the vehicle. This approach aligns perfectly with the stringent privacy and compliance requirements of ITS. Otherwise, in the real world, the

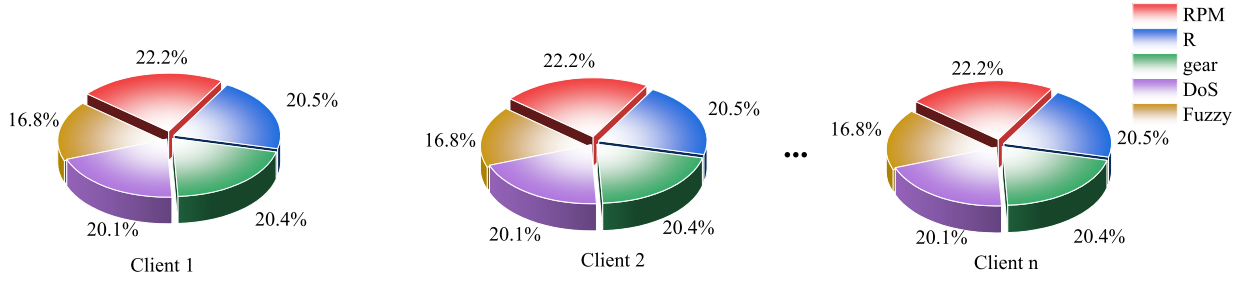


Fig. 5. Data sample distribution on each client under the IID setting.

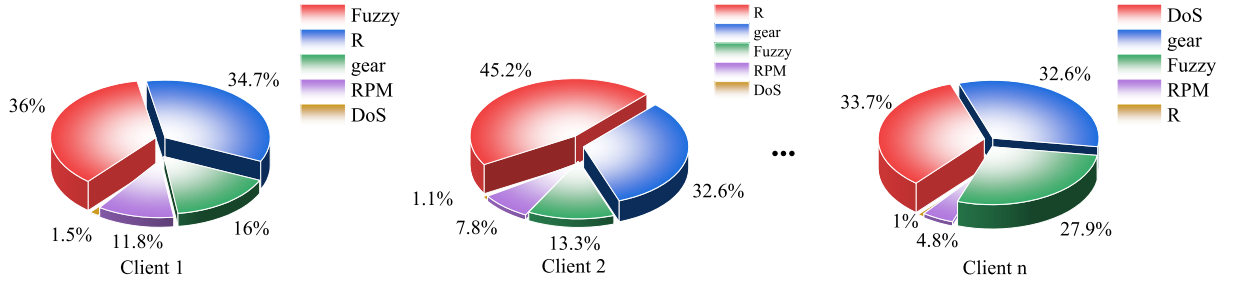


Fig. 6. Data sample distribution on each client under the non-IID setting.

data on each client may not follow an identical distribution. We consider two distinct scenarios: IID and non-IID. In the IID scenario, the samples in the dataset are independent and identically distributed, meaning that the data from each vehicle share similar distributions and categories. To achieve this, we employ stratified sampling to ensure that the data distribution across local vehicles aligns with the global dataset. This approach helps simulate a real-world scenario where multiple devices share similar data distributions, as illustrated in Fig. 5. To simulate a more challenging scenario, the data distribution in the Non-IID mode is randomly disturbed, resulting in discrepancies in the data distributions between vehicles. This makes model training more challenging for each vehicle. To address this, we introduce a dynamic non-IID sampling method, which incorporates an appropriate deviation factor to simulate the data heterogeneity that may exist between different vehicles in real-world settings, as shown in Fig. 6.

In our experiments, we set up 10 vehicles, with each vehicle independently training its local model and utilizing the global model's weights for training. In the first training round, each vehicle initializes the model parameters, while subsequent rounds utilize the aggregated global model parameters from the previous round. During each iteration process, local vehicles train their models based on their own data and then submitted the model updates to the global model. After each iteration, all local vehicles send their updated model weights to the server for weighted aggregation. Additionally, both local vehicles and the global model are trained for 25 iterations. During the model training process, the local models used by the vehicles are either trained based on knowledge distillation from a teacher model or through conventional training, with a multilayer perceptron (MLP) serving as the base architecture. And we use the adam optimizer with an initial learning rate set to 0.001. The batch size is set to 32, which was

determined based on empirical values from multiple tests conducted during the experiments. Meanwhile, we employ a dropout layer to prevent overfitting, with the dropout rate set to 0.2. To further prevent overfitting, L2 regularization is applied to the weights of all fully connected layers, with a regularization coefficient of $1e-4$.

To achieve optimal classification performance, we incorporate two loss functions during model distillation: Kullback-leibler (KL) divergence and categorical cross-entropy (CCE), which are combined to compute the total loss function, as formalized in Algorithm 1. KL divergence is primarily used in the knowledge distillation process. Specifically, during student model training, the KL divergence quantifies the discrepancy between the output probability distribution (softmax outputs) of the student model and the soft targets (probability distribution) generated by the teacher model [25], [26], [27]. CCE measures the difference between the model's predictions and the ground-truth labels in classification tasks [28], [29], [30]. The mathematical formulations of these loss functions are:

$$D_{KL}(P \parallel Q) = \sum_i P(x_i) \log \left(\frac{P(x_i)}{Q(x_i)} \right), \quad (10)$$

$$\text{distill_loss} = D_{KL} \left(\frac{P(t)}{T} \parallel \frac{P(s)}{T} \right), \quad (11)$$

$$\text{student_loss} = - \sum_{i=1}^c y_{\text{true},i} \log(y_{\text{student},i}), \quad (12)$$

$$\begin{aligned} \text{total_loss} &= \alpha \cdot \text{student_loss} + (1 - \alpha) \cdot \text{distill_loss} \\ &= \alpha \left(- \sum_{i=1}^c y_{\text{true},i} \log(y_{\text{student},i}) \right) \\ &\quad + (1 - \alpha) \left(D_{KL} \left(\frac{P(t)}{T} \parallel \frac{P(s)}{T} \right) \right). \end{aligned} \quad (13)$$

Eqs. (10) and (11) represent the KL divergence functions. Specifically, Eq. (10) denotes the KL divergence after temperature scaling, while Eq. (11) corresponds to the KL divergence without temperature scaling, commonly referred to as the distillation loss. The loss function used for the student model is defined in Eq. (12), based on the cross-entropy loss. The total loss function is described in Eq. (13). In these equations, i denotes the class index (e.g., class 1, class 2, etc.), $P(x_i)$ represents the probability predicted by the teacher model for class i , while $Q(x_i)$ denotes the corresponding probability predicted by the student model. $P(t)$ and $P(s)$ refer to the full probability distributions output by the teacher and student models, respectively. The parameter T is the temperature coefficient, which controls the smoothness of the output distributions during distillation. $y_{\text{true},i}$ indicates the ground-truth distribution for class i , typically expressed in one-hot encoding, and $\hat{y}_{\text{student},i}$ denotes the predicted probability for class i output by the student model.

Algorithm 2 Model Parameters Encryption

Input: Client 1 to n model parameters $W_{i,r}$, security parameters

Output: Encrypted model parameters $CT_{i,r}$

- 1: Vectorize parameters: $\bar{W}_{i,r} = \varphi(W_{i,r})$
 - 2: Blind: $\hat{W}_{i,r} = \bar{W}_{i,r} - \sum_{j=1}^N M_i^{(j,r)} + \sum_{j=1}^N M_j^{(i,r)}$
 - 3: Split the parameters into m random blocks $\{b_k^{(i,r)}\}$, ensuring linear reconstructability
 - 4: Use CRT to encrypt blocks: $CT_i = \sum_{k=1}^m b_k S_k T_k \bmod (S)$
 - 5: Return encrypted model parameters $CT_{i,r}$
-

F. Parameter Encryption

After completing local model training, client C_i obtains the local model parameters $W_{i,r}$ for round r . To protect privacy, each client encrypts its model parameters according to Algorithm 2. First, to convert original model parameters into integers for encoding, the transformation function $\varphi(x)$ is defined as follows:

$$\varphi(x) = \begin{cases} \lfloor x \times 10^t \rfloor, & \text{if } x \times 10^t - \lfloor x \times 10^t \rfloor < 0.5, \\ \lfloor x \times 10^t \rfloor + 1, & \text{otherwise,} \end{cases} \quad (14)$$

where t is a positive integer (e.g., $t = 5$), which controls the precision of the transformed model parameters. If the decimal part of $x \times 10^t$ is less than 0.5, the value is rounded down, otherwise, it is rounded up.

Then, blinding encoding is applied to enhance security. Specifically, random matrices $M_i^{(j,r)}$ and $M_j^{(i,r)}$ are used to perform weighted masking and shifting, ensuring that the original parameters cannot be directly inferred, even if ciphertexts are exposed.

Subsequently, random block partitioning is performed, splitting $\hat{W}_{i,r}$ into m random blocks $\{b_k^{(i,r)}\}$ while preserving linear reconstructability. This ensures that no single block contains complete parameter information, instead, all blocks must be combined to recover the original data.

To further enhance privacy and reduce storage overhead, each client employs CRT to encrypt each block. Specifically, given the following congruence system:

$$\begin{cases} CT_{1,r} \equiv b_1^{(i,r)} \pmod{p_1}, \\ CT_{2,r} \equiv b_2^{(i,r)} \pmod{p_2}, \\ \vdots \\ CT_{m,r} \equiv b_m^{(i,r)} \pmod{p_m}. \end{cases} \quad (15)$$

CRT ensures that each partial value b_k (the k -th parameter block from **Step 3** in Algorithm 2) can be uniquely embedded into the final ciphertext $CT_{i,r}$ while remaining recoverable during decryption. Leveraging CRT, the original model parameters are first encoded (i.e., converted into integers) and subsequently partitioned into multiple random smaller blocks. Each block can be encrypted independently and in parallel, enabling efficient computation on low-power multi-core platforms. This approach is particularly well suited for resource-constrained vehicular network scenarios. The encryption process can be formulated as:

$$CT_{i,r} = \sum_{k=1}^m b_k S_k T_k \bmod (S), \quad (16)$$

where b_k denotes the k -th parameter block generated in Step 3, S_k represents a modulus from the predefined modulus set S for constructing congruence equations, T_k is the multiplicative inverse or construction factor corresponding to modulus S_k that satisfies CRT conditions, and S ($S = \prod_{k=1}^m p_k$) serves as the composite modulus enabling all encrypted blocks to be incorporated into a unified congruence system.

G. Global Model Aggregation

Upon receiving all model upload transactions from local clients, blockchain validators must first verify the validity of the submitted transactions. Subsequently, an aggregator is randomly selected to perform the aggregation operation, as detailed in Algorithm 3. It primarily consists of the following three steps:

Step 1: Transaction validation. The system verifies the integrity of each block to detect potential malicious attacks or tampering. Any unauthorized modification to a block will alter its corresponding hash value, which can be identified by examining the complete blockchain and verifying the *previous_hash* values.

Step 2: Aggregator selection and operation. After all local clients upload their locally trained model parameters, the designated aggregator (central server) retrieves these encrypted model parameters from the blockchain and computes their aggregated value.

Step 3: Block generation. The aggregated parameters for round r of the specific model training task are encrypted and packaged into a new block, denoted as *block_r*, which is then appended to the blockchain.

H. Local Model Updating

During the update phase, each client C_i extracts the aggregated parameters from *block_r* and executes the decryption

Algorithm 3 Model Aggregation**Input:** Encrypted model parameters stored in the blockchain

$$\{CT_{i,r}\}_{i=1}^N$$

Output: New block $block_r$

- 1: Verify the transactions uploaded by local clients
- 2: Randomly select an aggregator from all nodes
- 3: Compute the aggregation value: $w_r = \sum_{i=1}^N CT_{i,r}$
- 4: Encrypt the aggregation parameter w_r and generate new block $block_r$
- 5: Return the new block $block_r$

Algorithm 4 Decryption and Update**Input:** Aggregated and encrypted model parameter w_r **Output:** Decrypted model parameter W_r

- 1: Decrypt aggregated parameter w_r : $B_k^{(r)} = w_r \bmod p_k$
- 2: Convert $B_k^{(r)}$ from finite field to integers using:
- 3: $f^{-1}(B_k^{(r)}) = \begin{cases} B_k^{(r)} - p_k, & p_k/2 \leq B_k^{(r)} < p_k \\ B_k^{(r)}, & 0 \leq B_k^{(r)} < p_k/2 \end{cases}$
- 4: Sum all transformed values: $\bar{W}_r = \sum_{k=1}^m B_k^{(r)}$
- 5: Compute final parameters: $W_r = \frac{\bar{W}_r}{10^f \times N}$
- 6: **Return** W_r

protocol specified in Algorithm 4. Firstly, the initial decryption step applies modular reduction to ciphertext w_r , producing discrete values $\{B_k^{(r)}\}_{k=1}^m$ through the operation:

$$B_k^{(r)} = w_r \bmod (p_k) \quad (17)$$

where the set $\{p_k\}$ consists of pre-selected pairwise distinct large prime numbers, which are employed to implement CRT-based encryption/decryption mechanism.

Then, negative value handling is performed. Through the function f^{-1} , map $B_k^{(r)} \in [0, p_k)$ to the symmetric interval $[-\frac{p_k-1}{2}, \frac{p_k-1}{2}]$, supporting the aggregation of negative values (e.g., model parameters may be negative).

Sum all transformed values $\hat{B}_k^{(r)}$ to obtain $\bar{W}_r$, then divide by $10^f \times N$ to get the aggregated model parameters W_r for round r . The calculation formulas can be obtained from:

$$\bar{W}_r = \sum_{k=1}^m \hat{B}_k^{(r)}, \quad (18)$$

$$W_r = \frac{\bar{W}_r}{10^f \times N}, \quad (19)$$

where 10^f is the quantization factor and N is the number of local clients.

IV. EXPERIMENT AND EVALUATION

A. Environment Setup

The proposed method was implemented utilizing several python libraries, including *scikit-learn*, *base64*, *rdflib*, *numpy*, *hashlib*, *tensorflow*, *pytorch*, and *seaborn*. All experiments were conducted on a high-performance workstation equipped with an intel (R) core (TM) i7-14700HX processor (20 physical cores, 28 logical threads, base frequency of 2.10 GHz)

TABLE I
CONFUSION MATRIX

		True category	
		1	0
Predicted category	1	True positive (TP)	False positive (FP)
	0	False negative (FN)	True negative (TN)

and 64 GB of RAM. The system operated on a 64-bit Microsoft Windows 10 Professional (version 22H2, build 19045.3803) with UEFI firmware and a polestar Z790 M-ATX D4 motherboard. The hardware platform, identified as device *XS-20250502KFBO*, provided sufficient computational capacity for model training and evaluation under realistic experimental conditions.

B. Dataset

The datasets utilized in this work include **Car-Hacking**, **CICIoV2024**, and **CICIDS2017**, each serving distinct purposes in the evaluation of IDS and vehicular network security. A detailed overview of each dataset is provided as follows:

Car-Hacking Dataset: This dataset primarily contains attack behavior data from vehicular networks, making it a crucial resource for evaluating IDS in the context of IoV. This dataset is collected from a simulated vehicular network environment, which includes both normal traffic and various attack types, such as denial of service (DoS) attacks, spoofing, and unauthorized access. The Car-Hacking dataset provides an invaluable resource for assessing the resilience of IDS models in detecting and mitigating security threats within the automotive network environment.

CICIoV2024 Dataset: This dataset is specifically designed for research in the security of vehicular networks, with a focus on testing the effectiveness and robustness of intrusion detection systems in an IoV context. It includes a wide array of network traffic data, along with various attack behaviors associated with vehicular communication systems. This dataset is particularly useful for evaluating IDS performance under real-world conditions, where vehicular networks face a range of sophisticated and evolving attack vectors.

CICIDS2017 Dataset: This dataset, released by the Canadian Institute for Cybersecurity (CIC), is widely regarded as one of the standard datasets for network traffic intrusion detection. It includes both normal network traffic and numerous types of cyber-attacks, such as brute force attacks, botnet traffic, and DoS attacks. This dataset is extensively used to benchmark IDS algorithms and has become a key resource in the field of network security, facilitating the evaluation of detection systems in a variety of attack scenarios.

C. Evaluation Metrics

To evaluate the detection performance, we defined a confusion matrix as shown in Table I.

In Table I, *TP* denotes the number of network flows that are actually anomalous and correctly predicted as anomalous. *FN* represents the number of network flows that are actually anomalous but incorrectly predicted as normal. *FP* indicates

the number of network flows that are actually normal but erroneously predicted as anomalous. TN corresponds to the number of network flows that are both actually normal and correctly predicted as normal.

Specifically, based on this confusion matrix, we define several metrics to assess their ability to predict whether a network flow is normal or anomalous. The following performance indicators are used to evaluate the classification effectiveness:

$$ACC = \frac{TP + TN}{TP + TN + FP + FN}, \quad (20)$$

where accuracy (ACC) represents the proportion of correctly predicted packets to the total number of packets. Higher values indicate better performance. While accuracy provides an overall indication of the classifier's performance, it does not fully capture the model's effectiveness when dealing with imbalanced datasets or different types of classification errors. Following accuracy, precision (P) focuses specifically on the positive predictions made by the model.

$$P = \frac{TP}{TP + FP}, \quad (21)$$

where P measures the impact of misclassified instances on correctly classified instances. In addition to precision, recall (R) provides insight into how well the model detects actual anomalies.

$$R = \frac{TP}{TP + FN}, \quad (22)$$

where R is the ratio of correctly predicted attacks to the actual attack traffic. Finally, the $F1$ -score combines precision and recall into a single metric.

$$F1 - score = 2 \cdot \frac{P \cdot R}{P + R}, \quad (23)$$

where the $F1$ -score represents the harmonic mean of precision and recall, serving as a comprehensive performance metric.

Moreover, we comprehensively evaluate key performance metrics such as model complexity, the number of trainable parameters, and memory consumption, which are specifically defined as follows: providing quantitative guidance for model selection in practical applications.

Model complexity: This reflects the structural intricacy of a model, typically quantified by metrics such as the number of network layers, neuron count (for neural networks), or decision rules (for tree-based models). Higher model complexity may enhance representational capacity but can also lead to overfitting and increased computational overhead.

Trainable parameters: This refers to the total number of learnable weights and bias terms in a model. A larger parameter count generally implies stronger fitting capability but significantly raises computational resource demands and training time costs. The parameter count for each layer is calculated as:

$$P_{layer} = (N_{in} + 1) \cdot N_{out}, \quad (24)$$

where N_{in} is the input dimension (i.e., the number of neurons in the previous layer), and N_{out} is the output dimension (i.e., the number of neurons in the current layer). Then, the total number of parameters is the sum of all layer parameters:

$$P_{total} = \sum_{layer} P_{layer}. \quad (25)$$

Memory consumption: This quantifies the dynamic memory usage during model training and inference, primarily influenced by parameter scale, batch processing data size, and intermediate activation values. Excessive memory requirements may hinder the model's deployability on resource-constrained devices. Assuming each parameter (including weights and biases) occupies 4 bytes, the memory consumption is calculated as:

$$M = P_{total} \cdot 4. \quad (26)$$

These metrics collectively form a balanced evaluation framework for model performance and computational efficiency.

D. Evaluation Results

The proposed method adopts a lightweight model design, which reduces memory consumption and the number of parameters by simplifying the neural network architecture. In addition, knowledge distillation is utilized. By leveraging the soft labels provided by the teacher model, the local model can effectively learn the teacher's knowledge and achieve relatively high accuracy, thereby achieving a balance between computational load and detection accuracy.

We evaluate our method on three datasets and observe high performance. Specifically, on the Car-Hacking and CICIoV2024 datasets, the model achieves an accuracy exceeding 99%. On the CICIDS2017 dataset, the accuracy reaches over 97%, as shown in Table II. The differences in detection accuracy can be attributed to two factors: differences in datasets and data distribution. Car-Hacking and CICIoV2024 datasets primarily simulate various attack types targeting vehicles, with a focus on security issues in in-vehicle networks. Their attack patterns are relatively concentrated, resulting in better model performance on these datasets. CICIDS2017 dataset contains external network traffic data, which has a wider variety of attacks and encompasses a broader range of network security threats. Its complexity and the diversity of attacks may cause the model to perform slightly worse on certain attack categories, particularly in terms of intrusion detection accuracy. As well as, the difference in data distribution is another important factor contributing to performance fluctuations. In practical applications, the data from vehicular devices and networks are often not independent and identically distributed (non-IID). As a result, the model may not generalize well due to the large variability between the data, leading to a decrease in accuracy. Furthermore, we calculate the number of parameters and memory consumption of the local model and compare it with the baseline models.

According to Eq.(25) to Eq.(26), the number of parameters in the proposed model on the Car-Hacking, CICIoV2024, and CICIDS2017 datasets is 9,989, 9,861, and 11,904, respectively, with corresponding memory consumption of 0.038 MB, 0.038 MB, and 0.045 MB. In contrast, MLP model exhibits parameter counts of 44,293, 44,228, and 48,007 on the same datasets, with memory consumption of 0.169 MB, 0.168 MB, and 0.183 MB, respectively, as illustrated in Fig. 7.

Typically, the traditional MLP consists of at least three hidden layers with 256, 128, and 64 neurons, whereas our

TABLE II
COMPARISON OF METHODS ON THREE DATASETS

Method	Car-Hacking dataset			CICIoV2024 dataset			CICIDS2017 dataset		
	Accuracy	Parameter	Memory(MB)	Accuracy	Parameter	Memory(MB)	Accuracy	Parameter	Memory(MB)
CNN [31]	99.90%	124,421	0.4746	100.00%	124,356	0.4744	97.51%	124,551	0.4751
MLP [32]	99.96%	44,293	0.169	100.00%	44,228	0.168	97.74%	48,007	0.183
LSTM [33]	99.95%	511,045	1.9495	99.90%	510,980	1.9492	96.10%	511,175	1.95
Transformer [34]	99.98%	3,204,357	12.224	99.99%	3,204,228	12.223	99.40%	3,267,335	12.464
Proposed Method	99.10%	9,989	0.038	99.65%	9,861	0.038	97.10%	11,904	0.045

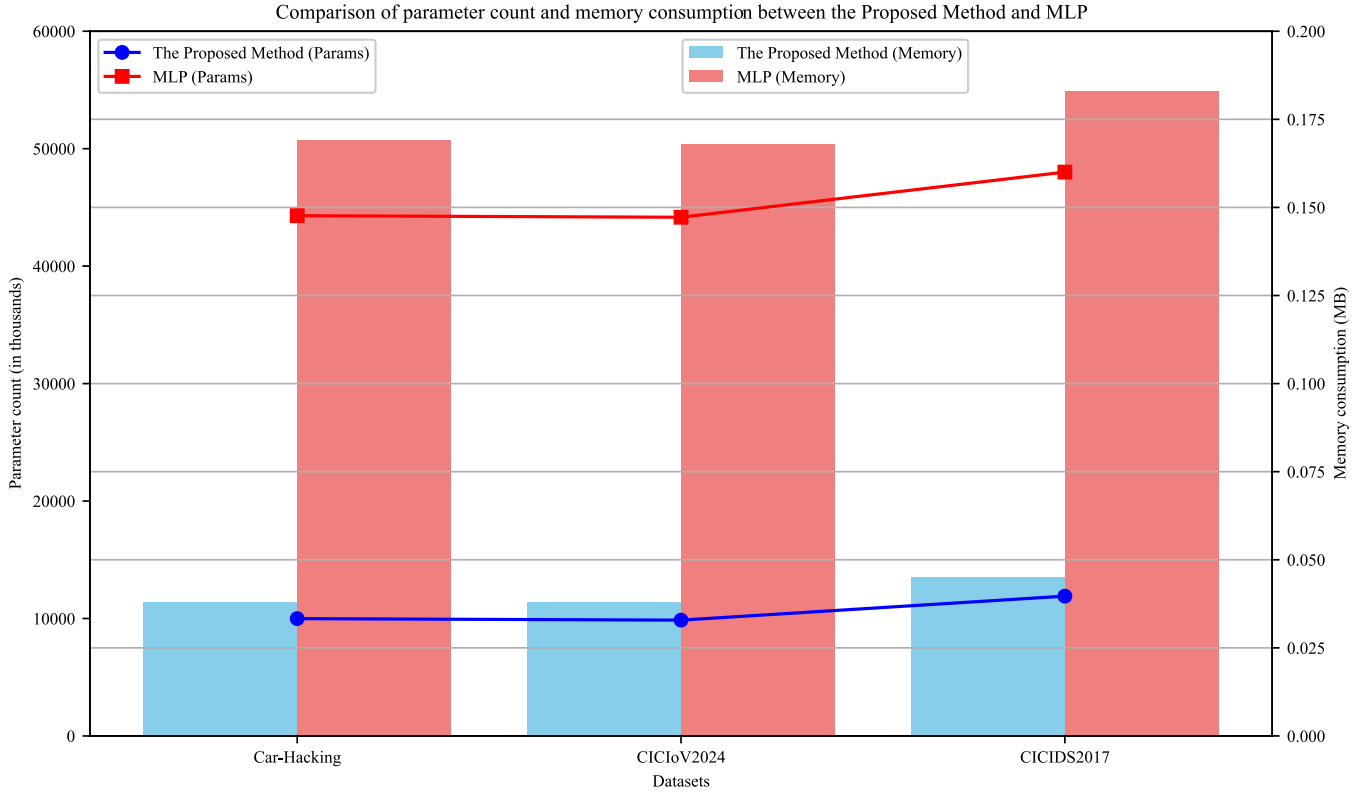


Fig. 7. Comparison of model paramers and memory usage.

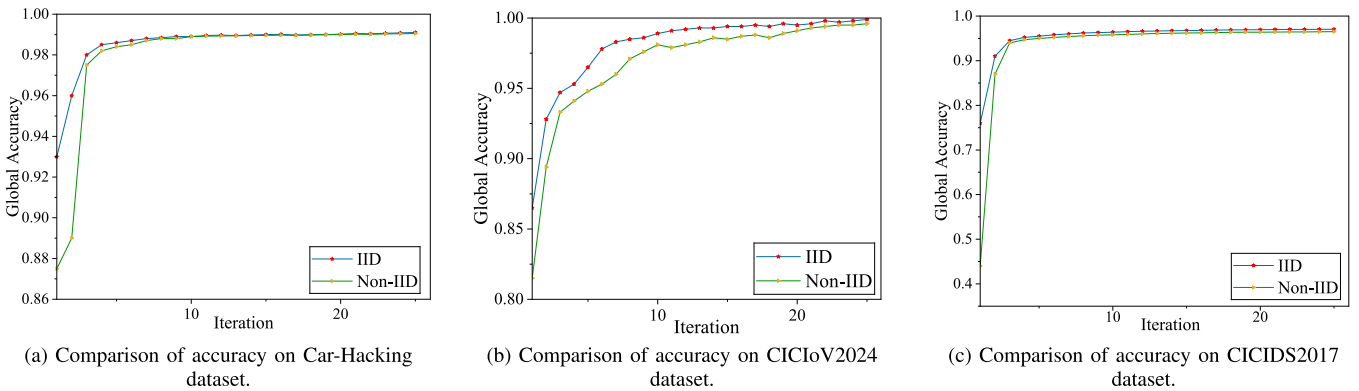
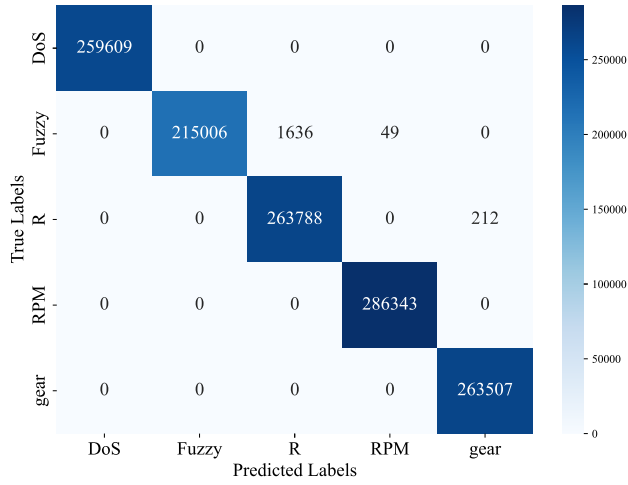


Fig. 8. Accuracy comparison of the proposed method on three datasets under IID and non-IID conditions.

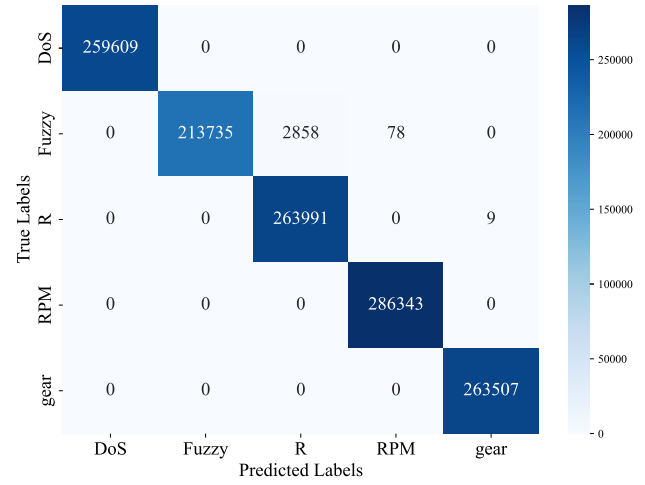
proposed lightweight model consists of only two layers with 128 and 64 neurons. Although the proposed model exhibits lower accuracy than the conventional MLP across all three datasets, it significantly reduces the number of parameters

and memory usage to approximately one-quarter of that of the baseline model.

To further evaluate the classification performance, Figs. 9–11 present the heatmaps of the proposed model on

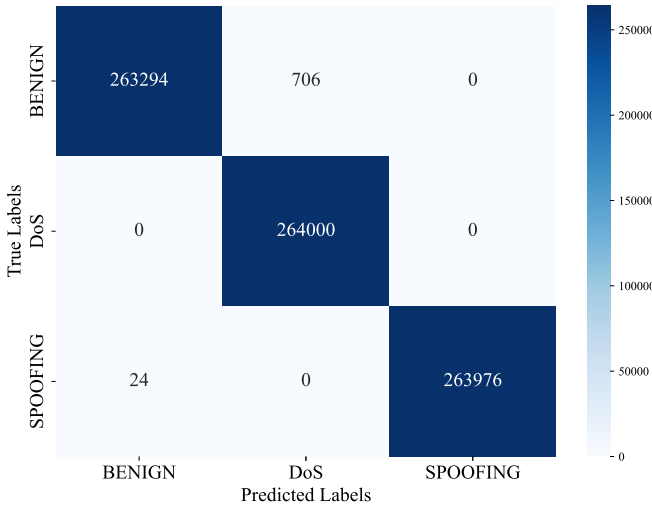


(a) Proposed method under IID.



(b) Proposed method under non-IID.

Fig. 9. Heatmap on multi-class classification on the Car-Hacking dataset.

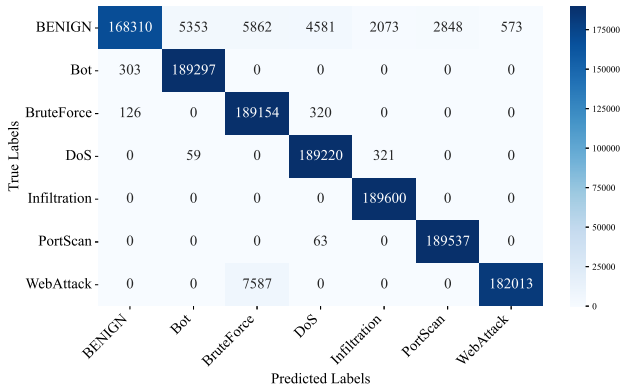


(a) Proposed method under IID.

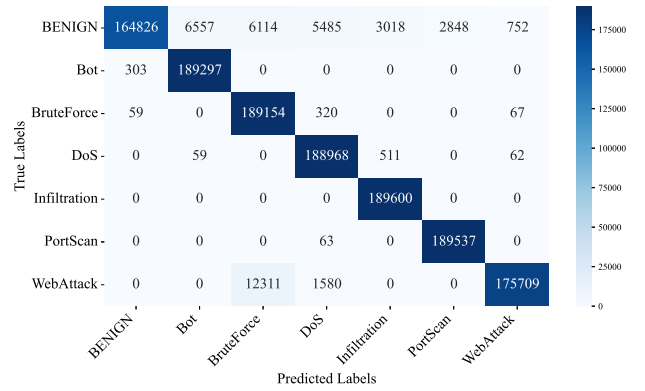


(b) Proposed method under non-IID.

Fig. 10. Heatmap on multi-class classification on the CICIoV2024 dataset.



(a) Proposed method under IID.



(b) Proposed method under non-IID.

Fig. 11. Heatmap on multi-class classification on the CICIDS2017 dataset.

the three datasets in the process of model training, which are essentially confusion matrices. The horizontal axis (Predicted

Labels) represents the categories predicted by the model, while the vertical axis (True Labels) indicates the true labels.

TABLE III

PERFORMANCE EVALUATION ON DIFFERENT DATASETS UNDER IID AND NON-IID CONDITIONS

Dataset	IID			non-IID		
	P	R	F1-score	P	R	F1-score
Car-Hacking	99.86%	99.83%	99.84%	99.78%	99.73%	99.75%
CICIoV2024	99.91%	99.91%	99.91%	99.88%	99.88%	99.88%
CICIDS2017	97.82%	97.73%	97.70%	97.13%	96.98%	96.94%

Each cell in the matrix reflects the relationship between predicted and actual categories. Values along the diagonal represent correctly classified samples, whereas off-diagonal values indicate misclassifications. The color intensity of each cell corresponds to the magnitude of the value. From the heatmaps, it is evident that the proposed IDS model achieves satisfactory classification results across all datasets.

Considering that client data in real-world scenarios is often non-IID, we conduct comparative experiments under both IID and non-IID settings. As shown in Fig. 8, the accuracy under the non-IID setting is initially lower than that under the IID setting, but both scenarios converge to high accuracy after several rounds of training. This demonstrates that the proposed scheme is well-suited not only for IID but also for more realistic non-IID environments, highlighting its robustness and practical applicability. To enhance the comprehensiveness of the experimental evaluation, we add the evaluation metrics defined in Eq.(21) to Eq.(23) (i.e., precision (P), recall (R), and F1-score), as shown in Table III. The model achieves precision of 99.86%, recall of 99.83%, and F1-score of 99.84% on the Car-Hacking dataset, precision of 99.91%, recall of 99.91%, and F1-score of 99.91% on the CICIoV2024 dataset, and precision of 97.82%, recall of 97.73%, and F1-score of 97.70% on the CICIDS2017 dataset. Under the non-IID scenario, the model achieves precision of 99.78%, recall of 99.73%, and F1-score of 99.75% on the Car-Hacking dataset, precision of 99.88%, recall of 99.88%, and F1-score of 99.88% on the CICIoV2024 dataset, and precision of 97.13%, recall of 96.98%, and F1-score of 96.94% on the CICIDS2017 dataset. The evaluation metrics above indicate that, with IID data, the distribution is more uniform, leading to better training performance and more stable results.

Moreover, to comprehensively evaluate the performance of the proposed method, we conduct a comparative analysis against several mainstream deep learning models, including CNN, long short-term memory networks (LSTM), MLP, and transformer architectures. As shown in Table II, the comparison spans multiple dimensions of multi-class classification tasks, such as classification accuracy, the number of model parameters, and memory consumption. Next, we will provide a detailed explanation of the settings for the comparison models.

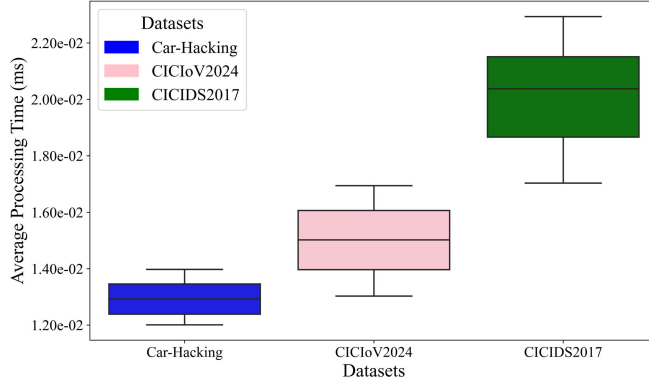
The CNN model performs feature extraction and classification through convolutional layers and fully connected layers. The first convolutional layer has 256 neurons, followed by two additional convolutional layers with 128 and 64 neurons, using the ReLU activation function. A maxPooling1D layer is used to reduce the feature map size, and the final classification is done using a fully connected layer with a softmax activation function. The model is trained using the adam optimizer

with a categorical cross-entropy loss function, a batch size of 128, and 100 epochs. The LSTM model consists of three LSTM layers with 256, 128, and 64 neurons, respectively, to capture temporal dependencies in sequential data. ReLU activation is applied to the fully connected layers, and the output layer uses a softmax activation function for multi-class classification. To prevent overfitting, a 50% dropout layer is included. The model is trained using the adam optimizer with a categorical cross-entropy loss function, a batch size of 128, and 100 epochs. The transformer model, suitable for sequence data classification, includes multi-head self-attention layers and position-wise feed-forward networks. The input is first mapped to a 64-dimensional embedding space through a dense layer, followed by several transformer blocks. Information is aggregated through a global average pooling layer, and a dense layer with a ReLU activation function and a dropout layer are applied to prevent overfitting. The output layer uses a softmax activation function. The model is trained using the adam optimizer with sparse categorical cross-entropy loss, a batch size of 32, and 100 epochs.

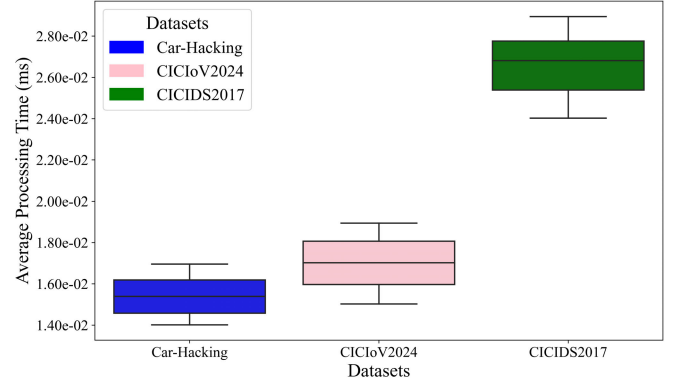
Experimental results demonstrate that although our method exhibits slightly lower accuracy than certain deep learning models on specific datasets, it significantly outperforms them in terms of parameter efficiency and memory utilization. In the Car-Hacking dataset, the parameter count of our approach is only 8.01% of that of CNN, 22.55% of MLP, 1.95% of LSTM, and merely 0.31% of the transformer. On the CICIoV2024 dataset, the parameter count accounts for just 7.93% compared to CNN, 22.30% of MLP, 1.93% of LSTM, and 0.31% of the transformer. Similarly, on the CICIDS2017 dataset, our method requires only 9.56% of the parameters of CNN, 24.80% of MLP, 2.31% of LSTM, and 0.36% of transformer. However, this lightweight design slightly reduces the accuracy. To select the appropriate model configuration based on different application requirements, we balance the following factors. For applications that require high accuracy, more complex models can be chosen, although this will increase memory and computational burdens. To this end, we adjust the MLP model architecture and conduct relevant experiments for validation. The adjusted MLP consists of three hidden layers, with 128, 128, and 64 neurons in each layer. In both IID and non-IID scenarios, the accuracy on the Car-Hacking dataset reaches 99.85% and 99.80%, respectively; on the CICIoV2024 dataset, the accuracy is 99.98% and 99.97%, and on the CICIDS2017 dataset, the accuracy is 98.10% and 97.96%. Compared to the 128-64 MLP, this adjustment leads to significant accuracy improvements. Specifically, the accuracy on the Car-Hacking dataset increases by approximately 0.7%, on the CICIoV2024 dataset by about 0.3%, and on the CICIDS2017 dataset by around 1%. Besides, we compare the average processing time per packet under different MLP architectures, as shown in Fig. 12(a) displays the average processing time per packet for the 128-64 MLP model architecture: 0.013 ms on the Car-Hacking dataset, 0.015 ms on the CICIoV2024 dataset, and 0.020 ms on the CICIDS2017 dataset. Fig. 12(b) presents the average processing time per packet for the 128-128-64 MLP model architecture: 0.015 ms on the Car-Hacking dataset, 0.017 ms on the CICIoV2024

TABLE IV
COMPARISON OF THE NUMBER OF PARAMETERS AND ACCURACY UNDER DIFFERENT MLP ARCHITECTURES

Model	Car-Hacking Dataset			CICIoV2024 Dataset			CICIDS2017 Dataset		
	Parameter	Memory (MB)	Accuracy	Parameter	Memory (MB)	Accuracy	Parameter	Memory (MB)	Accuracy
MLP(128-64)	9,989	0.038	99.85%	9,861	0.038	99.80%	11,904	0.045	98.10%
MLP(128-128-64)	26,501	0.101	99.98%	26,771	0.101	99.97%	28,423	0.108	97.96%



(a) Average processing time per data packet on MLP model with 128 and 64 neurons.



(b) Average processing time per data packet on MLP model with 128, 128 and 64 neurons.

Fig. 12. Average processing time per data packet.

TABLE V

COMPARISON OF ENCRYPTION AND DECRYPTION TIME FOR DIFFERENT METHODS IN ONE FL ROUND WITH MLP(128-64)

Method	Encryption time (ms)	Decryption time (ms)
CRT	19.93	19.02
RSA [35]	24.83	41.13
HE [36]	35.14	215.30

TABLE VI

COMPARISON OF ENCRYPTION AND DECRYPTION TIME FOR DIFFERENT METHODS IN ONE FL ROUND WITH MLP(128-128-64)

Method	Encryption time (ms)	Decryption time (ms)
CRT	31.92	31.91
RSA [35]	65.86	112.43
HE [36]	65.43	408.46

dataset, and 0.026 ms on the CICIDS2017 dataset. Compared to the 128-64 MLP, the average processing time per packet increased by 0.02 ms, 0.02 ms, and 0.06 ms on the three datasets, respectively. Similarly, we also compared the number of parameters and memory consumption, as shown in Table IV. In terms of the number of parameters, the adjusted 128-128-64 MLP model had 26,501, 26,371, and 28,423 parameters on the Car-Hacking, CICIoV2024, and CICIDS2017 datasets, respectively, with memory consumption of 0.101 MB, 0.101 MB, and 0.108 MB. Compared to the 128-64 MLP, the number of parameters increase by 16,512, 16,510, and 16,519, and memory consumption increased by 0.063 MB, thereby increasing the computational burden on onboard resources.

In addition, in our proposed scheme, the encryption and decryption method based on CRT is compared with two widely used cryptographic RSA and HE in terms of encryption and decryption time performance, as shown in Table V.

The CRT-based method demonstrates superior efficiency in both encryption and decryption phases, with an encryption

time of only 19.93 ms and a decryption time of 19.02 milliseconds. In contrast, RSA exhibits a clear disadvantage in processing time, requiring 24.83 ms for encryption and 41.13 ms for decryption. In addition, HE exhibits significantly higher computational overhead due to its inherent complexity, with encryption and decryption times of 35.14 milliseconds and 215.30 ms, respectively. These results clearly indicate that HE may introduce substantial performance costs in such application scenarios compared to CRT and RSA. Similarly, the encryption and decryption time vary depending on the MLP model architecture. Therefore, we conduct relevant experiments to evaluate the impact of encryption methods on system latency. As shown in Table VI, the encryption and decryption time for CRT are 31.92 ms and 30.9 ms, respectively, for RSA, they are 65.86 ms and 112.43 ms, and for HE, the encryption and decryption times are 65.42 ms and 408.46 ms, respectively. It is evident that, apart from CRT, the other two encryption methods have relatively high encryption and decryption times. To ensure that the encryption and decryption time do not exceed the system's latency requirements, we adopt a strategy of compressing the model parameters in this paper. This approach reduces the parameter size, thereby lowering the computational overhead during encryption and decryption. This ensures that the system can meet the stringent real-time requirements of ITS while maintaining high accuracy.

These findings strongly indicate the potential of the proposed method for deployment in resource-constrained vehicular environments, such as in-vehicle systems within intelligent transportation networks.

V. CONCLUSION

To protect vehicular privacy data, this paper proposed a lightweight intrusion detection method for IoV based on FL

and blockchain technology. In this framework, each vehicle trained a model locally and shares only model parameters, thereby significantly reducing the risk of data leakage. Moreover, blockchain was employed to prevent malicious participants from tampering with locally trained model parameters. In addition, knowledge distillation was also utilized to compress the model, leading to substantial reductions in parameter count and memory consumption for local devices. Experiments conducted on the Car-Hacking, CICIoV2024, and CICIDS2017 datasets demonstrated that, compared with baseline models, the proposed approach achieved a parameter size of only 9989, 9861, 11904 and a memory footprint of 0.039, 0.038, 0.045 MB. The relevant code has been uploaded to GitHub for readers to replicate the experimental results: zhajiawei666/Lightweight-Privacy-Preserving-IDS-for-ITS. In future work, we plan to further improve the model architecture to enhance detection accuracy on the CICIDS2017 dataset.

REFERENCES

- [1] H. Sedjelmaci, N. Kaaniche, A. Boudguiga, and N. Ansari, "Secure attack detection framework for hierarchical 6G-enabled Internet of Vehicles," *IEEE Trans. Veh. Technol.*, vol. 73, no. 2, pp. 2633–2642, Feb. 2024.
- [2] X. Li et al., "Cognitive AmBC-NOMA IoV-MTS networks with IQI: Reliability and security analysis," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 2, pp. 2596–2607, Feb. 2023.
- [3] W. Jiang and X. Lv, "A distributed Internet of Vehicles data privacy protection method based on zero-knowledge proof and blockchain," *IEEE Trans. Veh. Technol.*, vol. 73, no. 5, pp. 6332–6345, May 2024.
- [4] S. Almutlaq, A. Derhab, M. M. Hassan, and K. Kaur, "Two-stage intrusion detection system in intelligent transportation systems using rule extraction methods from deep neural networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 12, pp. 15687–15701, Dec. 2023.
- [5] M. Fu, P. Wang, M. Liu, Z. Zhang, and X. Zhou, "IoV-BERT-IDS: Hybrid network intrusion detection system in IoV using large language models," *IEEE Trans. Veh. Technol.*, vol. 74, no. 2, pp. 1909–1921, Feb. 2025.
- [6] Y. Wu et al., "Intrusion detection for Internet of Things: An anchor graph clustering approach," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 1965–1980, 2025.
- [7] S. Anbalagan, G. Raja, S. Gurumoorthy, R. D. Suresh, and K. Dev, "IIDS: Intelligent intrusion detection system for sustainable development in autonomous vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 12, pp. 15866–15875, Dec. 2023.
- [8] A. Chougule, I. Kulkarni, T. Alladi, V. Chamola, and F. R. Yu, "HybridSecNet: In-vehicle security on controller area networks through a hybrid two-step LSTM-CNN model," *IEEE Trans. Veh. Technol.*, vol. 73, no. 10, pp. 14580–14591, Oct. 2024.
- [9] X. Lin et al., "ByCAN: Reverse engineering controller area network (CAN) messages from bit to byte level," *IEEE Internet Things J.*, vol. 11, no. 21, pp. 35477–35491, Nov. 2024.
- [10] H. J. Jo and W. Choi, "A survey of attacks on controller area networks and corresponding countermeasures," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 7, pp. 6123–6141, Jul. 2022.
- [11] X.-Y. Kong and G.-H. Yang, "An intrusion detection method based on self-generated coding technology for stealthy false data injection attacks in train-ground communication systems," *IEEE Trans. Ind. Electron.*, vol. 70, no. 8, pp. 8468–8476, Aug. 2023.
- [12] Y. Xue, J. Pan, Y. Geng, Z. Yang, M. Liu, and R. Deng, "Real-time intrusion detection based on decision fusion in industrial control systems," *IEEE Trans. Ind. Cyber-Phys. Syst.*, vol. 2, pp. 143–153, 2024.
- [13] L. Wei, J. Yang, H. Jin, J. Cui, J. Li, and D. He, "Sustainable learning-based intrusion detection system for VANETs," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 7, pp. 10080–10091, Jul. 2025.
- [14] L. Yang, A. Moubayed, and A. Shami, "MTH-IDS: A multitiered hybrid intrusion detection system for Internet of Vehicles," *IEEE Internet Things J.*, vol. 9, no. 1, pp. 616–632, Jan. 2022.
- [15] Y. Lu, Y. Guo, S. Chen, and J. Li, "An intrusion detection model based on deep convolutional factorization machine for controller area network bus in Internet of Vehicles," *IEEE Internet Things J.*, vol. 11, no. 22, pp. 36203–36213, Nov. 2024.
- [16] I. Mohammed Sayem, M. Islam Sayed, S. Saha, and A. Haque, "ENIDS: A deep learning-based ensemble framework for network intrusion detection systems," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 5, pp. 5809–5825, Oct. 2024.
- [17] T. Zhang et al., "Hybrid transfer and self-supervised learning approaches in neural networks for intelligent vehicle intrusion detection and analysis," *IEEE Internet Things J.*, vol. 12, no. 7, pp. 7677–7692, Apr. 2025.
- [18] Y. Wu, L. Nie, S. Wang, Z. Ning, and S. Li, "Intelligent intrusion detection for Internet of Things security: A deep convolutional generative adversarial network-enabled approach," *IEEE Internet Things J.*, vol. 10, no. 4, pp. 3094–3106, Feb. 2023.
- [19] J. Cui et al., "Collaborative intrusion detection system for SDVN: A fairness federated deep learning approach," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 9, pp. 2512–2528, Sep. 2023.
- [20] S. Liu et al., "Delay and energy-efficient asynchronous federated learning for intrusion detection in heterogeneous industrial Internet of Things," *IEEE Internet Things J.*, vol. 11, no. 8, pp. 14739–14754, Apr. 2024.
- [21] J. Zhang, C. Luo, M. Carpenter, and G. Min, "Federated learning for distributed IIoT intrusion detection using transfer approaches," *IEEE Trans. Ind. Informat.*, vol. 19, no. 7, pp. 8159–8169, Jul. 2023.
- [22] O. Aouedi, K. Piamrat, G. Müller, and K. Singh, "Federated semisupervised learning for attack detection in industrial Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 19, no. 1, pp. 286–295, Jan. 2023.
- [23] L. Jin et al., "Intrusion detection for future ITS: Integrated knowledge graph and artificial intelligence," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 10, pp. 14680–14689, Oct. 2025.
- [24] J. Zhang, J. Cui, H. Zhong, Z. Chen, and L. Liu, "PA-CRT: Chinese remainder theorem based conditional privacy-preserving authentication scheme in vehicular ad-hoc networks," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 2, pp. 722–735, Mar. 2021.
- [25] S. Ji, Z. Zhang, S. Ying, L. Wang, X. Zhao, and Y. Gao, "Kullback–Leibler divergence metric learning," *IEEE Trans. Cybern.*, vol. 52, no. 4, pp. 2047–2058, Apr. 2022.
- [26] F. E. Tosun, A. M. H. Teixeira, J. Dong, A. Ahlén, and S. Dey, "Kullback–Leibler divergence-based observer design against sensor bias injection attacks in single-output systems," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 2763–2777, 2025.
- [27] X.-X. Ren, G.-H. Yang, and X.-G. Zhang, "Historical data-based stealthy attack design on cyber-physical systems under Kullback–Leibler divergence," *IEEE Trans. Ind. Informat.*, vol. 20, no. 4, pp. 6664–6672, Apr. 2024.
- [28] Y. Ho and S. Wookey, "The real-world-weight cross-entropy loss function: Modeling the costs of mislabeling," *IEEE Access*, vol. 8, pp. 4806–4813, 2020.
- [29] H. N. Monday et al., "Enhancing ECG classification in cardiac diagnostics: A novel approach using adaptive focal cross-entropy loss function," *IEEE J. Biomed. Health Informat.*, vol. 29, no. 10, pp. 7161–7174, Oct. 2025.
- [30] C. Sun, H. Li, M. Song, and S. Hong, "A ranking-based cross-entropy loss for early classification of time series," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 8, pp. 11194–11203, Aug. 2024.
- [31] A. R. Javed, S. U. Rehman, M. U. Khan, M. Alazab, and T. G. Reddy, "CANintelliIDS: Detecting in-vehicle intrusion attacks on a controller area network using CNN and attention-based GRU," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1456–1466, Apr. 2021.
- [32] S. Cherfi, A. Lemouari, and A. Boulaiche, "MLP-based intrusion detection for securing IoT networks," *J. Netw. Syst. Manage.*, vol. 33, no. 1, p. 20, Jan. 2025.
- [33] Y. Yu, X. Zeng, X. Xue, and J. Ma, "LSTM-based intrusion detection system for VANETs: A time series classification approach to false message detection," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 12, pp. 23906–23918, Dec. 2022.
- [34] Z. Zeng, B. Zhao, X. Deng, X. Liu, J. Zheng, and J. Chen, "Toward intelligent attack detection with causal transformer in Internet of Things," *IEEE Internet Things J.*, vol. 12, no. 8, pp. 10751–10767, Apr. 2025.
- [35] H. Zhang et al., "Efficient and secure outsourcing scheme for RSA decryption in Internet of Things," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 6868–6881, Aug. 2020.
- [36] S. Reddi et al., "Privacy-preserving electronic medical record sharing for IoT-enabled healthcare system using fully homomorphic encryption, IOTA, and masked authenticated messaging," *IEEE Trans. Ind. Informat.*, vol. 20, no. 9, pp. 10802–10813, Sep. 2024.



Jiawei Zha received the B.E. degree from Xuzhou University of Technology, Xuzhou, China, in 2023. He is currently pursuing the Ph.D. degree with the School of Information Science and Technology, Nantong University, Nantong, China. His current research focuses on intrusion detection in vehicular networks.



Guoan Zhang (Senior Member, IEEE) received the B.S. degree in precision instruments, the M.S. degree in automatic instruments and equipment, and the Ph.D. degree in communication and information systems from Southeast University, Nanjing, China, in 1986, 1989, and 2001, respectively. He is currently a Full Professor with the School of Electronics and Information, Nantong University, Nantong, China. His current research interests include cognitive wireless networks and vehicular ad hoc networks.



Li Jin received the B.S. degree in computer science and technology degree from Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2004, the M.S. degree in soft engineering from Southeast University, Nanjing, in 2007, and the Ph.D. degree in communication and information system from Nantong University, Nantong, China, in 2020. She is currently an Associate Professor with the School of Information Science and Technology, Nantong University. Her research interests include wireless communication, artificial intelligence, and

vehicular networks.



Shuping Dang (Senior Member, IEEE) received the B.Eng. (Hons.) degree in electrical and electronic engineering from The University of Manchester, Manchester, U.K., the B.Eng. degree in electrical engineering and automation from Beijing Jiaotong University, Beijing, China, in 2014, via a joint “2+2” Dual-Degree Program, and the D.Phil. degree in engineering science from the University of Oxford, Oxford, U.K., in 2018. He has joined Research and Development Center, Huanan Communication Company Ltd., after graduating from the University of Oxford and was a Post-Doctoral Fellow with the Computer, Electrical and Mathematical Science and Engineering Division, King Abdullah University of Science and Technology, Thuwal, Saudi Arabia. He is currently a Lecturer with the School of Electrical, Electronic and Mechanical Engineering, University of Bristol, Bristol, U.K. His research interests include 6G communications, wireless communications, wireless security, and machine learning for communications.



Wei Duan received the Ph.D. degree from Chonbuk National University, Jeonju, South Korea, in 2017. He is currently a Full Professor with Nantong University, Nantong, China. He has participated in the World Class University Project, sponsored by the National Research Foundation of Korea Grant funded by the Korean Ministry of Education Science and Technology as a Vice Head Researcher. He has authored more than 90 journal articles. His research interests include a variety of topics in the areas of wireless communications. He served as a Guest

Editor for IEEE INTERNET OF THINGS JOURNAL Special Issue on Near-Field Communications (NFC) in Internet of Everything, *IEEE Network* Special Issue on Near-Field Communications: Challenges and Opportunities in the Networking Landscape, and *China Communications* Special Issue on New Advances in Nonorthogonal Multiple Access; and a Lead Guest Editor for *Wireless Communications and Mobile Computing* Special Issue on Architectures, Challenges, and Opportunities within 6G Emerging Technologies. He is an Editor of *Frontiers in Communications and Networks*.